

INTEGRATED DIGITAL HOUSE RENTAL AND TAX MANAGEMENT SYSTEM

RHTMS

A single platform connecting property owners, tenants, housing officers, tax officers and administrators across the residential rental lifecycle.

What is RHTMS?

- RHTMS digitizes Addis Ababa's residential rental lifecycle: property registration, tenant registration, rental agreements, housing-office review and approval, service-fee and tax payment, contract-agreement issuance with QR verification, and compliance/penalty tracking.
- It replaces a manual, paper-based, multi-office process with one shared system where five kinds of users each see only what their role needs.
- **Who uses it:**
 - Property Owners register houses and create agreements • Tenants search listings and are registered onto agreements • Housing Officers review and approve • Tax Officers assess and collect rental tax • Super Admins configure policy and manage accounts.
- **Main benefit: submission, review, approval, payment, tax and termination all happen in one auditable system instead of scattered paperwork across offices.**

The Current Rental & Tax Context

- Residential rental registration and rental-income taxation in Addis Ababa are governed by real regulatory instruments used as the grounding for this project.
 - Housing Rent Control and Management Proclamation No. 1320/2016 — the underlying legal framework for rental regulation.
 - Directive No. 184/2025 (Addis Ababa Housing Development and Management Bureau) — the Residential Rent Control and Management Directive; source for the registration workflow and administrative penalties (Article 22).
 - Income Tax (Amendment) Proclamation No. 1395/2025 — referenced basis for rental income tax administered by the Justice Bureau.
- This workflow has historically been paper-based and office-bound: registering an agreement, applying penalties, and assessing/paying tax happen as separate, disconnected steps with no shared digital record.
- **RHTMS is a semester-project prototype that models this real workflow against one shared, auditable database.**

Problems RHTMS Addresses

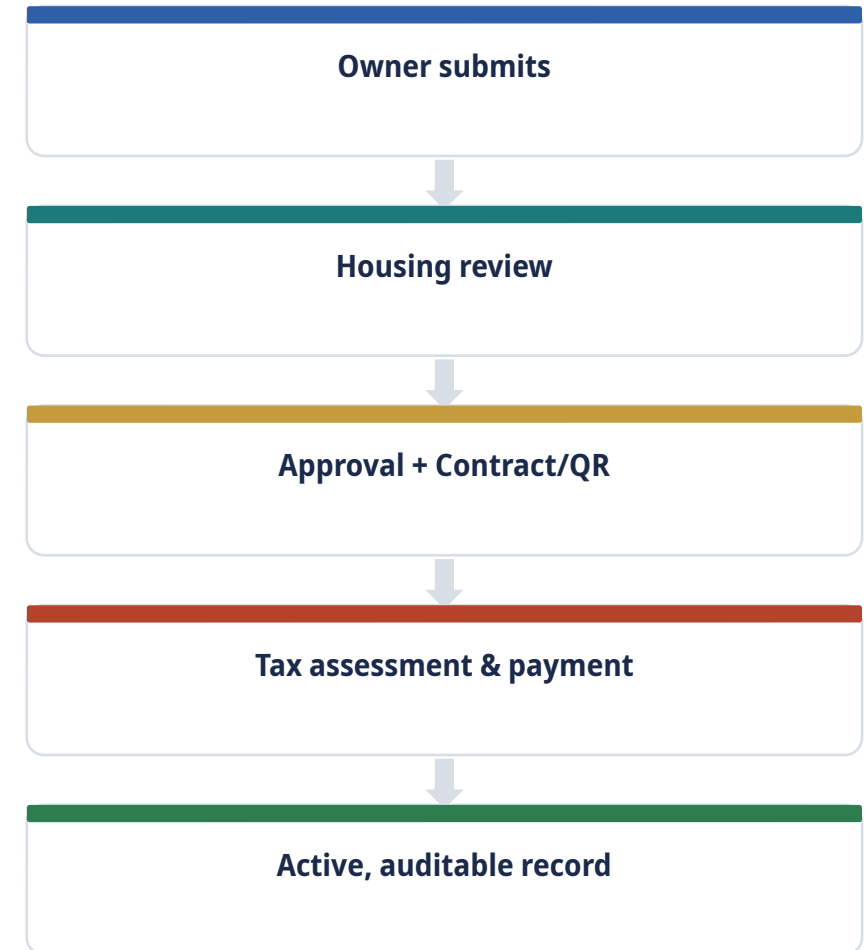
- Fragmented processes — registration, approval and taxation are handled by separate offices with no shared record.
- Manual paperwork — property, tenant and agreement details are recorded on paper, with no structured, searchable history.
- Delayed / unclear approvals — an owner or tenant has no reliable way to see where a submission stands.
- Lack of transparency in tax and penalty amounts — rules exist in directives but aren't applied consistently or visibly.
- Difficulty verifying rental agreements — no simple way for a third party to confirm a contract is genuine.
- Weak auditability — no consolidated trail of who approved, changed, or paid what, and when.

General & Specific Objectives

- **General objective**
 - Design and implement a single digital platform that manages the residential rental lifecycle — from property registration to tax compliance — connecting property owners, tenants, housing offices and tax offices under role-based access.
- **Specific objectives**
 - Allow property owners to register and manage properties and rental agreements digitally.
 - Allow tenants to search listed properties and be registered onto rental agreements.
 - Give housing officers a review/approval workflow for property registrations and agreements.
 - Give tax officers a way to assess and track rental-income tax against a configurable rate/rule, not hard-coded logic.
 - Generate a verifiable digital contract agreement (with QR code) once an agreement is approved.
 - Enforce role-based access control and record-level ownership so users can only act on their own records.
 - Maintain a full audit trail of approvals, payments and status changes for accountability.

How RHTMS Solves These Problems

- One shared, database-backed system replaces paper records for every office involved in the rental lifecycle.
- A configurable rules engine (tax rate, penalty amounts, price ranges) lets an administrator update policy without a code change — instead of guessed figures hard-coded into the system.
- Every submission carries an explicit status (Draft → Submitted → Under Review → Approved/Rejected → Active → Terminated) that owners, tenants and officers can all see.
- A QR-coded contract agreement lets anyone verify a rental contract's authenticity without contacting an office.
- An audit log and in-app notifications give every party visibility into what happened and when.



Who Uses RHTMS

Property Owner

Registers properties, creates rental agreements, pays service fees, views tax status.

Tenant

Searches listed properties, is registered onto agreements, pays rent-related fees, requests termination.

Housing Officer

Reviews/approves properties and agreements, reviews penalties, manages corrections and terminations.

Tax Officer

Assesses rental income tax, tracks overdue tax, reviews penalties, views payments.

Super Admin

Manages users, tax/penalty/price rules, system configuration, audit log and reports.

Public / Verifier

Anyone can scan a contract's QR code to confirm it is a genuine, system-issued agreement — no login required.

Layers & Technology Stack

User Interface

Next.js 16 App Router + React 19, Tailwind CSS v4, shadcn/ui (Base UI primitives), next-intl (English / Amharic / Afaan Oromoo)

Authentication & Authorization

Auth.js (NextAuth v5), credentials login + JWT sessions, central role-permission map re-checked on every action, plus record-level ownership checks

Application / Business Logic

Server Actions + framework-independent service modules (property, agreement, payment, tax, penalty, audit, notifications) validated with Zod

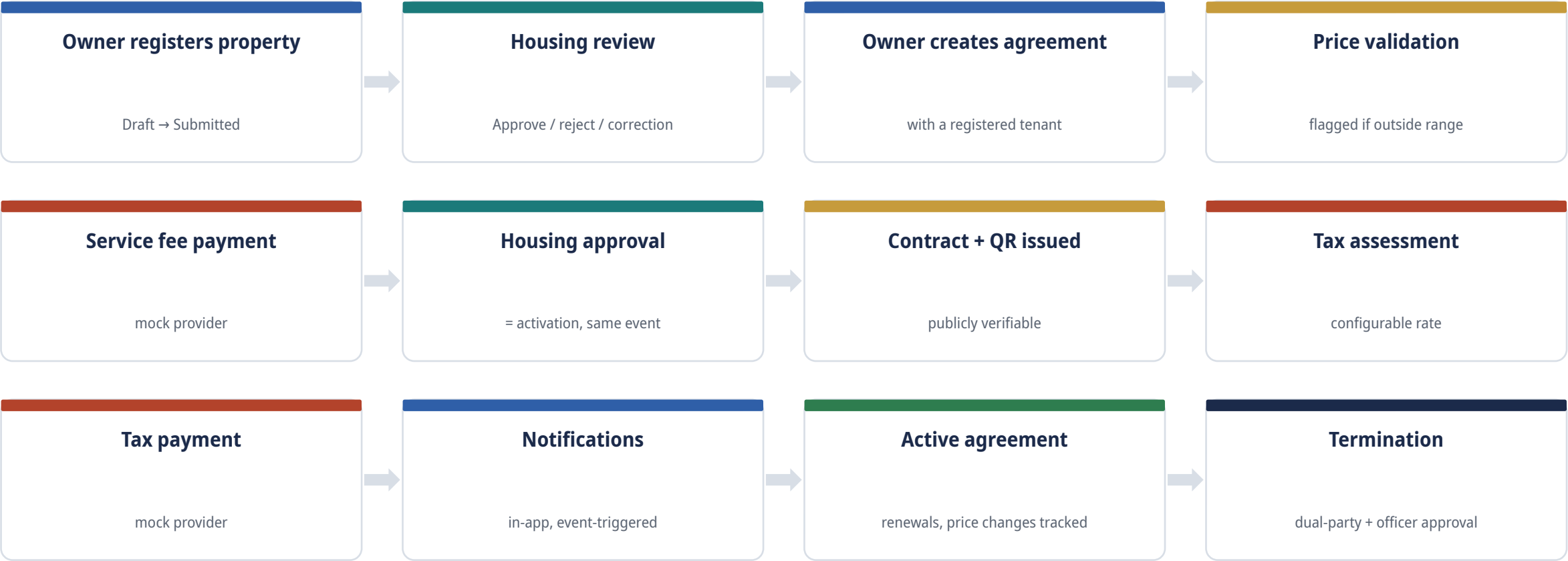
Database

PostgreSQL via Prisma 7 (driver-adapter client) — single schema, 21 models, the system's one source of truth

External / Mock Services

Multi-provider mock payment layer (CBE, Telebirr, Bank Transfer, Cash) behind a swappable PaymentProvider interface; QR generation for contract verification

End-to-End Lifecycle (as Implemented)



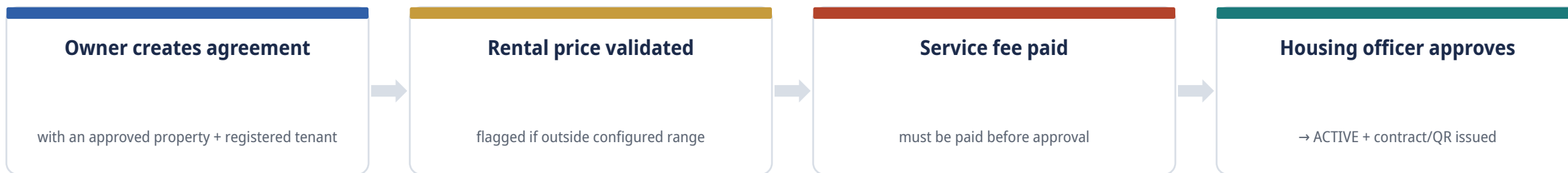
Note: housing-officer approval and activation are the same event in this implementation — there is no separate "approved but not yet active" state.

Owner → Housing Office → Approved Property



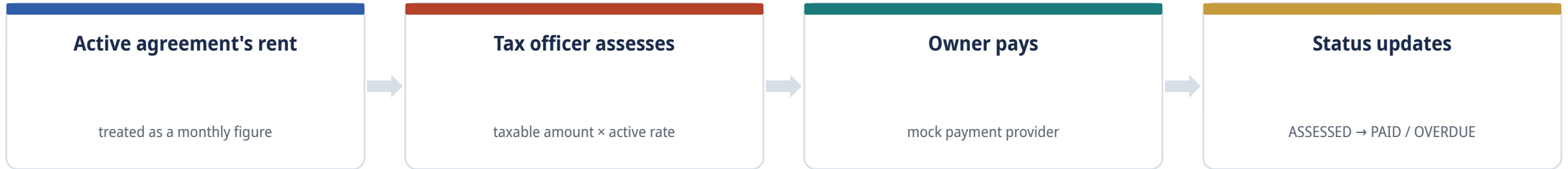
- If rejected or sent back for correction, the property returns to the owner, who can edit and resubmit — it is not a dead end.
- Only an APPROVED (or already ACTIVE) property can be used to create a rental agreement — enforced in the agreement-creation service, not just hidden in the UI.
- Approved properties become visible in public property search for tenants.
- Every review decision is recorded (PropertyReview) with the reviewing officer and reason, and written to the audit log.

Owner/Tenant → Validation → Active → Termination



- In this implementation the property owner creates the agreement directly for a tenant they've registered — a tenant-initiated "request to rent" flow is not yet built (see Limitations).
- Price changes and renewals are recorded as versioned history (AgreementVersion), not silent overwrites — the original terms stay visible.
- Termination requires an initiating party (owner or tenant) and housing-officer approval — it isn't unilateral.
- A cross-owner ownership check blocks any owner from creating, editing, or terminating another owner's agreement — verified by an automated negative test.

Assessment → Payment → Compliance Status



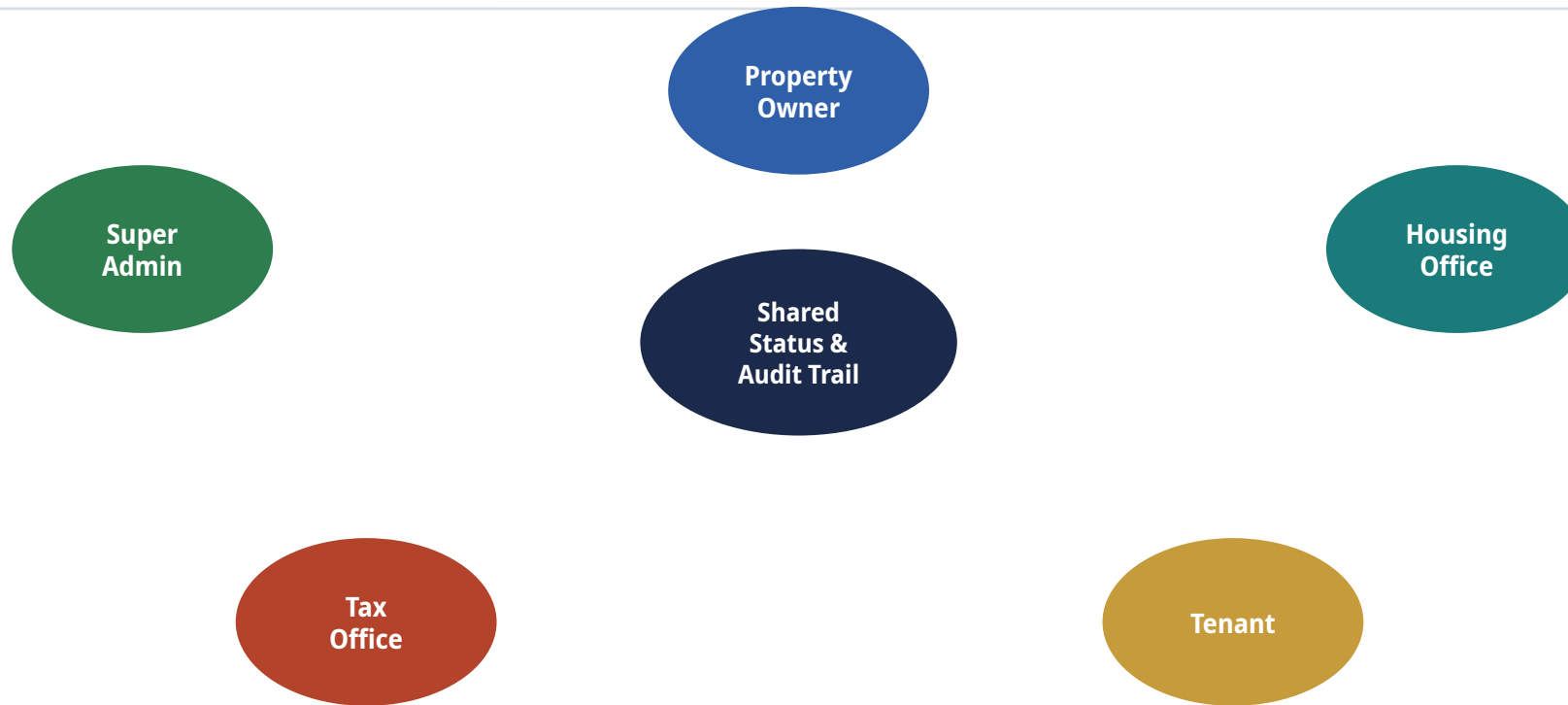
- Tax rate is a configurable TaxRule row (seeded at 11.5%), not a hard-coded constant — an admin can activate a new rate at any time, and historical assessments keep the rate that applied when they were made.
- *This 11.5% figure comes from the original project interview, not a verified statutory rate — Proclamation No. 1395/2025 does not specify an exact percentage in the material available to this project, so the rate is explicitly a configurable placeholder, not a legally verified number.*
- Overdue tracking uses a configurable grace period (default 30 days after the assessment period ends).
- Late-payment and compliance penalties (e.g. non-electronic rent payment) are grounded in Directive No. 184/2025, Article 22, expressed as a percentage of one month's rent.

Service Fee, Tax and Penalty Payments

PROTOTYPE / SIMULATED PAYMENT INTEGRATION — no real banking or mobile-money transaction ever occurs.

- Every payment type in the system — service fee, tax, penalty — goes through the same mock multi-provider abstraction: CBE, Telebirr, Bank Transfer and Cash.
- All four providers are mock/sandbox implementations that settle instantly behind a common PaymentProvider interface — a real integration could replace them without changing the rest of the application.
- Payments are recorded with status (INITIATED / COMPLETED / FAILED / REFUNDED), a reference number, and are tied to the specific agreement, tax assessment, or penalty they settle.
- Directive No. 184/2025 requires rent to be paid via bank or electronic means (Article 21(8)); non-electronic payment is modeled as a penalty (Article 22(7)) — the CASH option is kept only for classroom-demo flexibility.

How Offices Connect Through Shared Status



- Owner submits a property or agreement → status becomes SUBMITTED / UNDER_REVIEW, visible to Housing Office.
- Housing Office approves/rejects → the status change is what unlocks the next stage (agreement creation, contract issuance).
- Tax Office reads agreement/property data already in the system to assess tax — no re-entry of data between offices.
- Super Admin sets the policy (tax rate, penalty amounts, price rules) that both offices apply, and manages user accounts.

Defense in Depth

- Authentication — Auth.js (NextAuth v5) with credentials login (bcrypt-hashed passwords) and JWT sessions; no session table, the signed cookie is the session.
- Role-based access control — a central permission map (PERMISSIONS / ROLE_PERMISSIONS) is the single source of truth for what each of the 5 roles may do; every server action re-checks it rather than trusting the UI.
- Record-level authorization — ownership checks (e.g. assertOwnsProperty, assertPartyToAgreement) stop one owner from reading or writing another owner's property or agreement, verified by an automated negative test in the workflow-verification script.
- Route-level role enforcement — every dashboard section is additionally gated so an authenticated user of one role cannot open another role's dashboard section by navigating directly to its URL.
- Audit trail — every write to a sensitive record (approvals, payments, rule changes) is logged with actor, action, and before/after values.

Keeping Everyone Informed

- Implemented: in-app notifications, triggered automatically by real business events — property approval/rejection, agreement approval, payments, tax assessment, and penalty actions all create a Notification row for the affected user(s).
- Every dashboard shows a live unread-notification badge and a notifications page listing full history, with mark-as-read / mark-all-read actions.
- Every record carries an explicit status field (e.g. SUBMITTED, UNDER_REVIEW, APPROVED, ACTIVE, OVERDUE, TERMINATED) so a user can see exactly where their submission stands without contacting an office.
- *Not implemented: scheduled/proactive reminders (e.g. "your tax is due in 3 days") and SMS/email delivery — notifications are event-triggered and in-app only. This is an explicit, known scope boundary, not a bug (see Limitations).*

What the Super Admin Console Provides

- User management — view all accounts and roles across the system.
- Tax, penalty and price rule management — create a new active TaxRule / PenaltyRule / RentalPriceRule; the previous rule is deactivated, not deleted, preserving history for past assessments.
- System configuration — settings such as service-fee percentage and tax grace-period days.
- Audit log — a searchable record of sensitive actions across the system.
- Reports — revenue collected (tax, service fees, penalties), active-agreement counts, and a 6-month revenue trend chart, all computed live from the database, not static mock numbers.

Live figure

Tax Collected

Live figure

Service Fees Collected

Live figure

Penalties Collected

Live figure

Active Agreements

Figures shown on the real Admin → Reports page are computed on demand from actual payment/agreement records.

Insert From the Actual Running System

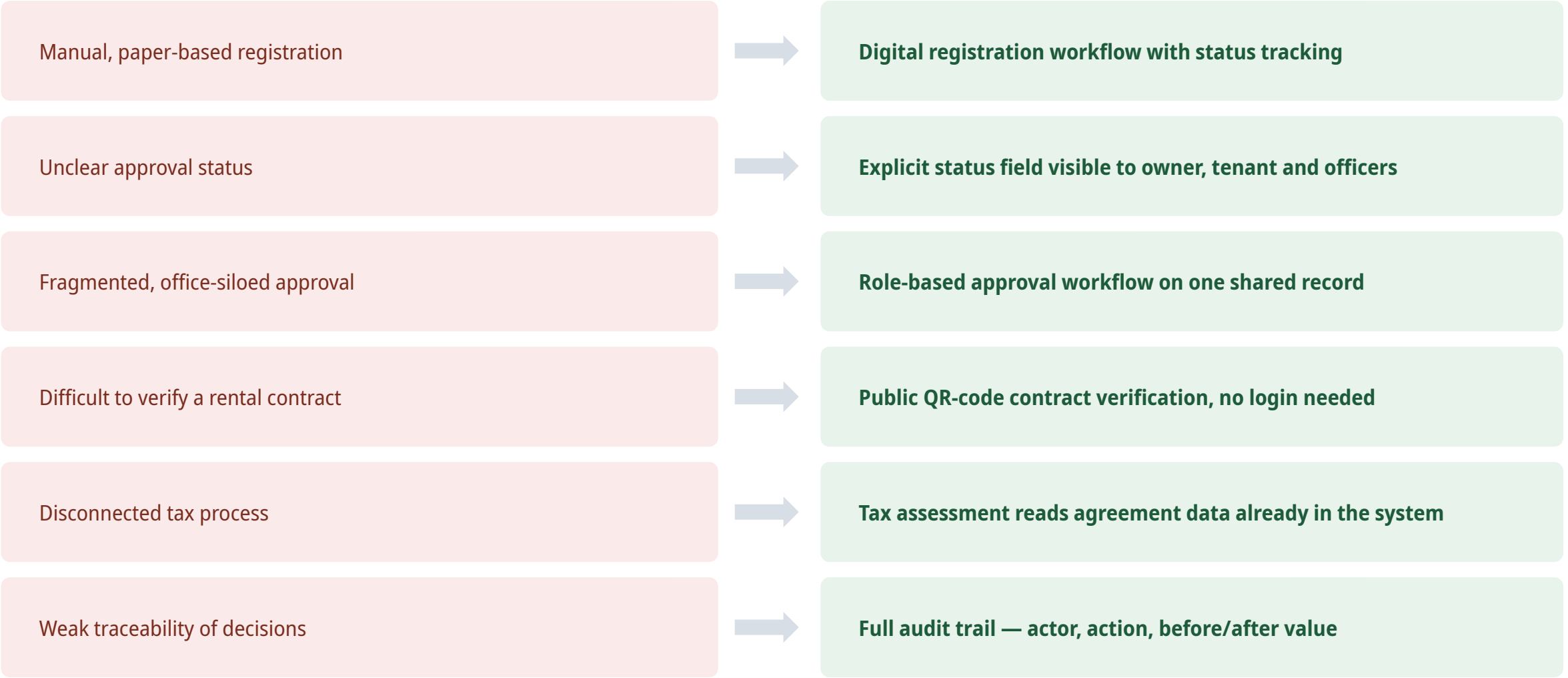
No browser-automation/screenshot tool was available this session — screenshots below are placeholders. Capture each from the running app (npm run dev) before presenting.

[screenshot] Home / public landing page	[screenshot] Login page	[screenshot] Owner dashboard	[screenshot] Property registration form	[screenshot] Housing officer — pending property review
[screenshot] Tenant — property search	[screenshot] Rental agreement detail + contract/QR	[screenshot] Tax officer dashboard	[screenshot] Payment page (mock provider selection)	[screenshot] Admin dashboard (reports/charts)

Functional Requirements vs. Implementation

Requirement	Status	Evidence / Feature
Property registration & housing review	Implemented	Owner submit → officer approve/reject/correction, status-gated
Tenant registration & search	Implemented	Self or owner-initiated registration; public property search
Rental agreements + price validation	Implemented	RentalPriceRule range check flags out-of-range rent
Service fee & tax payment	Simulated	Real workflow, mock CBE/Telebirr/Bank/Cash providers
Contract agreement + QR verification	Implemented	Public /verify/[token] page, no login required
Tax assessment (configurable rate)	Implemented*	*Rate is a configurable demo value (11.5%), not a verified statutory rate
Penalty rules & recording	Implemented	Grounded in Directive 184/2025, Art. 22 percentages
Agreement renewal / termination	Implemented	Dual-party request + officer approval, versioned history
Role-based access control	Implemented	Central permission map + per-request role enforcement
Notifications	Partial	In-app, event-triggered only — no SMS/email or scheduled reminders
Audit trail	Implemented	Actor, action, before/after value logged on sensitive writes
Admin reports & rule management	Implemented	Live revenue figures + tax/penalty/price rule console
Multi-language UI	Implemented	English / Amharic / Afaan Oromoo, 173 keys in sync
Document upload for properties	Not Implemented	Modeled in schema (PropertyDocument); no upload UI yet
Tenant-initiated "request to rent"	Not Implemented	Owner currently creates the agreement directly

Before → RHTMS



Verified Before This Presentation

4 / 4

Automated checks passed
(typecheck, lint, i18n, workflow)

16

Real end-to-end lifecycle steps
verified against the live database

5 / 5

Roles HTTP-traced across every
dashboard route (200, no errors)

1

Critical authorization bug found
and fixed before this presentation

- Automated: npm run typecheck, lint, verify:i18n and verify:workflow all pass — the workflow script runs a real 16-step lifecycle (property → review → agreement → service fee → approval/contract issuance → QR lookup → cross-owner block → tax assessment → tax payment → price-change history → penalty → renewal → termination → audit trail → history preserved) against the actual Postgres database.
- Manual/HTTP-level: every dashboard route for all 5 roles was requested with a real session cookie and returned HTTP 200 with correct content; logout correctly clears the session cookie; public QR verification correctly resolves genuine and bogus tokens.
- **Found and fixed: authenticated users of one role could open another role's dashboard section (e.g. a Tenant viewing the Admin dashboard) by navigating directly to its URL — a real cross-role authorization gap. Fixed by enforcing the role check in the request middleware, then re-verified across all role combinations plus the full automated suite with no regressions.**

Implemented vs. Prototype vs. Future

Implemented	Prototype / Mock	Partially Implemented	Future Enhancement
• Auth + role-based access control • Property registration & review • Rental agreements + price validation • Contract agreement + QR verification • Tax assessment (configurable rate) • Penalty recording & review • Renewal / termination workflow • In-app notifications • Full audit trail • Admin console & reports • English / Amharic / Afaan Oromoo UI	• Payments (CBE, Telebirr, Bank, Cash) — all simulated, instant settlement • Tax rate — configurable demo value, not a verified statutory rate • Identity verification — officer-set boolean flag, no live Fayda/National-ID API	• Notifications — event-triggered, in-app only (no SMS/email, no scheduled reminders)	• Property document upload UI (modeled in schema, not built) • Tenant-initiated "request to rent" flow • Automated/scheduled tests beyond the workflow script

What This Prototype Does Not (Yet) Do

- Payment integrations (CBE, Telebirr, bank transfer) are simulated — no real bank/mobile-money connection exists; a production deployment would need real gateway integration and PCI/financial compliance work.
- The rental-income tax rate (11.5%) is a configurable demo value from the original project interview, not confirmed against the exact text of Proclamation No. 1395/2025 — it must be verified/updated before any non-demo use.
- Identity verification is a manual officer decision (a checkbox), not a live government ID-verification API (e.g. Fayda).
- Notifications are in-app and event-triggered only — there is no SMS/email delivery and no scheduled reminder job (e.g. "tax due in 3 days").
- No production deployment/hosting infrastructure exists — this runs as a local development prototype.
- Property document upload (e.g. title deeds, ID scans) is modeled in the database schema but has no upload UI yet.
- Directive 184/2025 Article 22.8's escalating vacancy-penalty schedule is not implemented — it would require tracking how long an approved property sits without an agreement, which the current schema does not automate.

Where RHTMS Could Go Next

- Real payment-gateway integration (CBE Birr, Telebirr, bank APIs) behind the existing PaymentProvider interface — no rework of the payment logic itself needed.
- Real government identity verification (Fayda National ID) to replace the manual officer checkbox.
- SMS/email delivery and scheduled reminders for upcoming tax due-dates and rent due-dates.
- A tenant-initiated "request to rent" flow, as an alternative to owner-only agreement creation.
- Document upload for property titles/IDs, using the schema that already models it.
- A production deployment with proper hosting, backups and monitoring.
- A mobile application and advanced analytics/reporting for policy makers.

CONCLUSION

- The problem: Ethiopian residential rental registration and taxation have relied on fragmented, paper-based processes across separate housing and tax offices, with weak status visibility and auditability.
- The solution: RHTMS is a single, role-based platform that carries a property and its agreement from registration through housing approval, tax assessment, payment, and termination — on one shared, auditable record.
- What was verified: the full 16-step lifecycle, role-based access control (including a critical cross-role bug found and fixed during this final review), record-level ownership checks, QR-based contract verification, and multi-language UI — all confirmed against the running system, not assumed.
- Current result: a working, demonstrable prototype with real business logic and a real (mock-labeled) payment layer — production-ready in workflow design, not yet in external integrations.

RHTMS digitizes and connects the rental and tax management process — turning a fragmented, paper-based workflow into one accountable, role-based, auditable system.